 # MINI PROJECT - DAY 6

 # Title: Department Summary Report

STEP 1 - Linux Setup

mkdir day6_project

cd day6_project

Create employees.csv with the 8-row dataset

used in today's exercises.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's$ cd day6_project/
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ touch employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 18  2026 employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ cat > employees.csv << EOF
id,name,department,salary,city
1,Rahul,Sales,50000,Pune
2,Amit,IT,60000,Mumbai
3,Neha,HR,40000,Pune
4,Priya,IT,70000,Delhi
5,John,HR,30000,Mumbai
6,Ravi,Sales,45000,Bangalore
7,Sneha,IT,65000,Pune
8,Karan,Finance,55000,Delhi
EOF
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ cat employees.csv | column -t -s,
id  name   department  salary  city
1   Rahul  Sales       50000   Pune
2   Amit   IT          60000   Mumbai
3   Neha   HR          40000   Pune
4   Priya  IT          70000   Delhi
5   John   HR          30000   Mumbai
6   Ravi   Sales       45000   Bangalore
7   Sneha  IT          65000   Pune
8   Karan  Finance     55000   Delhi
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$
```

STEP 2 - Linux Pre-processing

Before touching Python, use Linux to:

1. Verify total row count (excluding header).

2. Extract and list unique departments.

3. Sort the file by department and save to

   sorted_by_dept.csv using sort.

Commands: wc -l cut sort uniq

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ cat employees.csv | wc -l
9
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ cut -d',' -f3 employees.csv | sort | uniq
Finance
HR
IT
Sales
department
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ sort -t',' -k3,3 employees.csv > sorted_by_dept.c
sv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ cat sorted_by_dept.csv
8,Karan,Finance,55000,Delhi
3,Neha,HR,40000,Pune
5,John,HR,30000,Mumbai
2,Amit,IT,60000,Mumbai
4,Priya,IT,70000,Delhi
7,Sneha,IT,65000,Pune
1,Rahul,Sales,50000,Pune
6,Ravi,Sales,45000,Bangalore
id,name,department,salary,city
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$
```

STEP 3 - Python Task

Create summary.py inside day6_project.

The program must:

1. Read employees.csv using open() and readlines().

2. Skip the header line.

3. For each department, calculate:

   - Total number of employees

   - Total salary

   - Average salary

   - Highest paid employee name and salary

4. Store all results in a nested dictionary.

5. Print a clean formatted report.

```
dictinary = {}

with open("employees.csv", "r") as file:
    data = file.readlines()
    for i in data[1:]:
        data = i.strip().split(',')

        name = data[1]
        dept = data[2]
        salary = int(data[3])

        if dept not in dictinary:
            dictinary[dept] = {
                "count": 0,
                "total_salary": 0,
                "highest_paid_name": "",
                "highest_paid_salary": 0
            }

        dictinary[dept]['count'] += 1
        dictinary[dept]['total_salary'] += salary

        if salary > dictinary[dept]['highest_paid_salary']:
            dictinary[dept]['highest_paid_salary'] = salary
            dictinary[dept]['highest_paid_name'] = name
```

```
                dictinary[dept]['highest_paid_name'] = name
print("Department Summary Report")
print("=====*5)
for key, value in dictinary.items():
    print(f"Department : {key}")
    print(f"Employees  : {value['count']}")
    print(f"Total sal  : Rs. {value['total_salary']}")
    print(f"Avg sal    : Rs. {round(value['total_salary'] / value['count']):,}")
    print(f"Top Earner : {value['highest_paid_name']} (Rs. {value['highest_paid_salary']:,})")
    print()
```

✓ 0.0s

Output :-

```
Department Summary Report
=========================
Department : Sales
Employees  : 2
Total sal  : Rs. 95000
Avg sal    : Rs. 47,500
Top Earner : Rahul (Rs. 50,000)

Department : IT
Employees  : 3
Total sal  : Rs. 195000
Avg sal    : Rs. 65,000
Top Earner : Priya (Rs. 70,000)

Department : HR
Employees  : 2
Total sal  : Rs. 70000
Avg sal    : Rs. 35,000
Top Earner : Neha (Rs. 40,000)

Department : Finance
Employees  : 1
Total sal  : Rs. 55000
Avg sal    : Rs. 55,000
Top Earner : Karan (Rs. 55,000)
```

STEP 4 - SQL Verification

Create the same employees table in SQLite.

Run the combined query from SQL Exercise 4.

Verify that SQL output matches

your Python report exactly.

This confirms your Python logic is correct.

```sql
-- Create the same employees table in SQLite.
-- Run the combined query from SQL Exercise 4.

CREATE TABLE TODAY_EMPLOYEE(
    ID NUMBER,
    NAME VARCHAR2(20),
    DEPARTMENT VARCHAR2(20),
    SALARY NUMBER,
    CITY VARCHAR2(20)
)
INSERT INTO TODAY_EMPLOYEE VALUES (1, 'Rahul', 'Sales', 50000, 'Pune');
INSERT INTO TODAY_EMPLOYEE VALUES (2, 'Priya', 'HR', 60000, 'Mumbai');
INSERT INTO TODAY_EMPLOYEE VALUES (3, 'Amit', 'IT', 45000, 'Pune');
INSERT INTO TODAY_EMPLOYEE VALUES (4, 'Neha', 'IT', 70000, 'Delhi');
INSERT INTO TODAY_EMPLOYEE VALUES (5, 'Karan', 'Sales', 55000, 'Bangalore');
INSERT INTO TODAY_EMPLOYEE VALUES (6, 'Sneha', 'HR', 52000, 'Pune');
INSERT INTO TODAY_EMPLOYEE VALUES (7, 'Rohan', 'IT', 48000, 'Mumbai');
INSERT INTO TODAY_EMPLOYEE VALUES (8, 'Divya', 'Sales', 62000, 'Delhi');
```

```sql
# Run the combined query from SQL Exercise 4.
# SQL Exercise 4 - Combined

# QUERY
# Write a single query that shows:
# - Department name
# - Number of employees
# - Average salary (rounded to 0 decimal places)
# - Highest salary
# - Lowest salary
#
# Only show departments with more than 1 employee.
# Order results by average salary descending.
#
# Concepts: COUNT, AVG, MAX, MIN, ROUND,
#           GROUP BY, HAVING, ORDER BY

SELECT DEPARTMENT,
    COUNT(*) AS NUMBER_OF_EMP,
    ROUND(AVG(SALARY), 0) AS AVERAGE,
    MAX(SALARY) AS HIGHEST_SALARY,
    MIN(SALARY) AS LOWER_SALARY
FROM TODAY_EMPLOYEE
GROUP BY DEPARTMENT
HAVING COUNT(*) > 1
ORDER BY AVERAGE DESC;
```